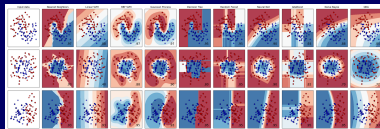


Recomendadores e interpretabilidad

Aprendizaje Automático · Sesión 4



Andrés Vásquez Restrepo

avasquezr3@eafit.edu.co

SI7009 - 1 (5553)

Universidad EAFIT

2026-2 – Medellín

Contenido

1. Del valor al ranking
2. La matriz de utilidad
3. Feedback explícito frente a implícito
4. Evaluar un recomendador
5. Filtrado colaborativo y factores latentes
6. ALS: Alternating Least Squares
7. Recomendadores híbridos
8. Interpretabilidad con SHAP
9. Hands-on y cierre

Dónde quedamos

Sesiones anteriores

- S1: baseline serio, Recall@k, validación sin leakage;
- S2: boosting y búsqueda de hiperparámetros;
- S3: el tiempo rompe la intercambiabilidad; walk-forward.

Ruta: ranking → contenido vs colaborativo → matriz de utilidad → implícito → evaluación → factorización → ALS → híbridos → SHAP → hands-on.

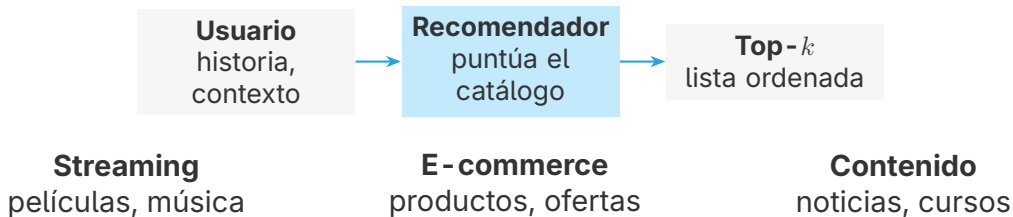
Idea clave: Hoy

Cada fila es un par **usuario-ítem** y el objetivo ya no es predecir un valor: es **ordenar** opciones para una persona.

Del valor al ranking: el problema de recomendar

Recomendar es ordenar opciones para una persona

¿Qué k ítems mostrar a **este** usuario, **ahora**?



Vocabulario mínimo

Término	Qué significa hoy
usuario u	una persona
ítem i	una película (o producto, canción, noticia)
interacción	u vio i y le gustó
score s_{ui}	número que el modelo asigna al par (u, i) : mayor = mostrar antes
top- k	los k ítems de mayor score que u no ha visto

Idea clave: Todo recomendador hace lo mismo

Asigna un score a cada par (usuario, ítem) y ordena. Lo que cambia entre métodos es **de dónde sale el score**.

El mini-mundo de la sesión

	<i>Toy Story</i>	<i>Lion King</i>	<i>Aladdin</i>	<i>Terminator 2</i>	<i>Alien</i>	<i>Die Hard</i>
Ana	5	4	4	1	·	·
Beto	4	·	5	·	2	·
Carla	·	2	·	5	4	5
Diego	1	·	·	4	5	·
Elena	5	·	4	4	?	?

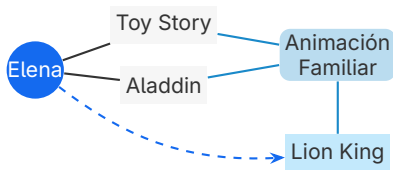
- ratings de 1 a 5; · = no la ha visto;
- Ana y Beto: **familiares**;
- Carla y Diego: **acción**;
- Elena: un poco de ambos.

Idea clave: **La pregunta de toda la sesión**

¿Qué película le recomendamos a **Elena**: Lion King, Alien o Die Hard?

Dos familias de recomendadores

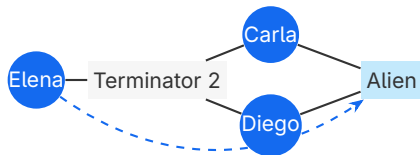
Basado en contenido



mira la **ficha** de las películas
no mira a otros usuarios

«te gustaron animadas → otra animada»

Filtrado colaborativo



mira **quién vio qué**
las películas son solo IDs

«quienes vieron lo mismo que tú también vieron esta»

La diferencia está en la evidencia, no en el algoritmo

	Basado en contenido	Filtrado colaborativo
Usa	atributos del ítem (géneros, año, actores, texto)	la matriz usuario $\times$ ítem
¿Mira a otros usuarios?	no	sí, es su única fuente
¿Mira el contenido?	sí	no
Elena recibe	Lion King (más animación)	Alien (fans de T2 la vieron)
Película nueva	se puede recomendar ya	imposible: nadie la vio
Riesgo	más de lo mismo	cold start, sesgo a lo popular

Idea clave: **Pregunta de control**

Si borra la columna de géneros, ¿cuál sigue funcionando? Si borra a los demás usuarios, ¿cuál sigue funcionando?

Dos formulaciones del mismo problema

	Rating prediction	Top - N ranking
Pregunta	¿qué nota le daría u al ítem i ?	¿qué k ítems mostrarle a u ?
Tipo	regresión sobre r_{ui}	ordenar ítems no vistos
Métrica	RMSE, MAE	Precision@k, NDCG@k
Datos	ratings explícitos	clics, vistas, compras

Idea clave: **En esta sesión**

Trabajamos con **top-N**: es lo que usa la industria y lo que un usuario realmente ve.

Qué se optimiza además del acierto

Objetivo	Pregunta
Relevancia	¿los ítems recomendados le interesan al usuario?
Cobertura	¿qué fracción del catálogo llega a recomendarse?
Novedad	¿recomendamos algo que el usuario no habría encontrado solo?
Diversidad	¿la lista tiene variedad o diez versiones de lo mismo?

Un sistema que solo maximiza relevancia termina recomendando siempre lo mismo a todos.

El baseline que hay que vencer

Más popular

$$\text{score}(i) = \#\{u : r_{ui} \text{ observado}\}$$

score de la película i = **cuántas personas la vieron**. No aparece u : no depende de quién pide la recomendación $\Rightarrow$ **la misma lista para todos** (menos lo que cada uno ya vio).

Aleatorio

Un ítem cualquiera del catálogo: piso mínimo.

Ejemplo: Mini - mundo

Vistas: TS 3 · Aladdin 3 · T2 3 · Alien 2 · LK 1 · DH 1.

Elena recibe Alien (bien).

Diego, fan de acción, recibe Toy Story y Aladdin: ignora sus gustos.

Sorpresa frecuente

Aun así es **muy** difícil de superar: lo popular concentra gran parte de lo que la gente ve después.

La matriz de utilidad

La matriz de utilidad $R \in \mathbb{R}^{m \times n}$

	ítems (n)					
usuarios (m)	5		4			1
		3			5	
	4			2		
			5		4	
		1				3

- fila = usuario u , columna = ítem i ;
- celda = utilidad observada r_{ui} (rating, clics, compras);
- Ω = conjunto de celdas observadas;
- las celdas grises **no son ceros**: son desconocidas.

El mini-mundo es una R de 5×6 ;
MovieLens, una de 6.040×3.706 .

Sparsity: casi toda la matriz está vacía

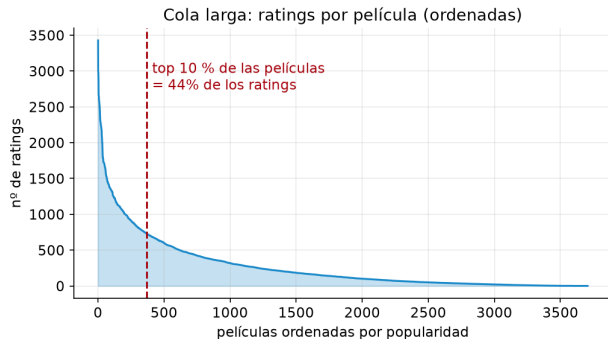
$$\text{densidad} = \frac{|\Omega|}{m \cdot n}$$

Dataset	Usuarios	Ítems	Densidad
Mini-mundo	5	6	$17/30 \approx 57 \%$
MovieLens 1M	6.040	3.706	$\approx 4,5 \%$
E-commerce típico	millones	cientos de miles	$< 0,1 \%$

Idea clave: **Consecuencia**

Un hueco no es un cero. Tratarlo como «no le gustó» sesga el modelo hacia lo que ya se mostró.

Cola larga y sesgo de popularidad



- pocos ítems concentran la mayoría de las interacciones;
- la mayoría del catálogo tiene muy pocas;
- los modelos tienden a recomendar lo popular y la cola larga queda invisible.

Cold start y cómo se almacena la matriz

Cold start

- **usuario nuevo:** no hay historia que comparar;
- **ítem nuevo:** nadie lo ha consumido todavía;
- el filtrado colaborativo no puede recomendar sin interacciones → motiva los **híbridos**.

Almacenamiento

user	item	value
1	1193	5
1	661	3
2	1357	4

Tripletas → matriz dispersa
`scipy.sparse.csr_matrix`: solo se guardan las celdas observadas.

Feedback explícito frente a implícito

Dos tipos de señal

	Explícito	Implícito
Ejemplos	ratings, likes, reseñas	clics, vistas, compras, tiempo
Volumen	escaso	abundante
Negativos	sí (una nota baja)	no hay negativos claros
Sesgo	solo opina quien quiere	depende de lo que se mostró
Lectura	cuánto le gustó	cuánta confianza tenemos en que le gustó

Idea clave: En producción

Casi todo el dato disponible es implícito.

Preferencia y confianza

Hu, Koren y Volinsky (2008) separan **qué** prefiere el usuario de **cuánto** le creemos:

$$p_{ui} = \begin{cases} 1 & \text{si } r_{ui} > 0 \text{ (interactuó)} \\ 0 & \text{si no} \end{cases} \qquad c_{ui} = 1 + \alpha r_{ui}$$

- p_{ui} : preferencia binaria (interactuó o no);
- c_{ui} : confianza, crece con la intensidad (vistas, repeticiones);
- α : cuánto pesa la intensidad (hiperparámetro).

Ejemplo: $\alpha = 40$

3 vistas: $p = 1$, $c = 121$.

Sin interacción: $p = 0$, $c = 1$.

Lo no observado cuenta como «probablemente no», con **baja** confianza.

De MovieLens explícito a implícito

Regla de conversión

- $r_{ui} \geq 4 \rightarrow$ interacción positiva;
- $r_{ui} \leq 3$ o sin rating $\rightarrow$ no observado.

Se conserva la fecha de cada interacción para el split temporal.

Qué se gana y qué se pierde

Se gana	Se pierde
un problema parecido a producción métricas top-N directas	los negativos explícitos (notas bajas) la intensidad del gusto

El mini-mundo en 0/1

	TS	LK	Ala	T2	Alien	DH
Ana	1	1	1	0	0	0
Beto	1	0	1	0	0	0
Carla	0	0	0	1	1	1
Diego	0	0	0	1	1	0
Elena	1	0	1	1	0	0

$p_{ui} = 1$ si $r_{ui} \geq 4$; el rating 1 de Ana en T2 pasa a 0.

Ejemplo: Confianza con $\alpha = 10$

Elena-Toy Story: $p = 1, c = 11$.

Elena-Alien: $p = 0, c = 1$.

El 0 de Alien dice «probablemente no», con poca seguridad. **No** dice «no le gusta».

Como aquí cada interacción vale 1, $c_{ui} = 1 + \alpha$ en lo visto y 1 en lo demás. Con conteos (3 vistas) sería $1 + 3\alpha$.

Trampas del feedback implícito

Missing not at random

No haber visto un ítem no significa que no le guste: puede que nunca se le mostró.

Feedback loop

El recomendador decide qué se muestra, y lo que se muestra genera los datos con los que se reentrena. Lo popular se vuelve más popular.

Respuesta práctica: medir cobertura y novedad además del acierto, y comparar siempre contra el baseline de popularidad.

Pausa: verifique

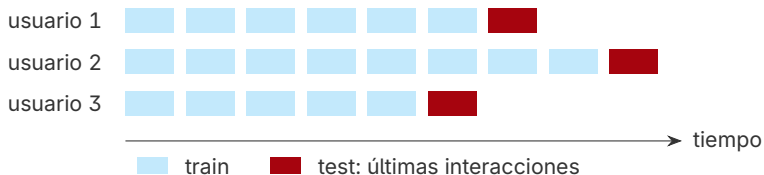
Taller / tarea: Tres preguntas, dos minutos

1. En el mini-mundo, ¿el 0 de Elena en Alien significa que no le gustaría?
2. ¿Qué recomendador puede usar la columna de géneros y cuál no la necesita?
3. Llega *Frozen* (Animación, Familiar) y nadie la ha visto: ¿quién puede recomendarla hoy?

Respuestas: no, es desconocido · contenido / colaborativo · solo contenido.

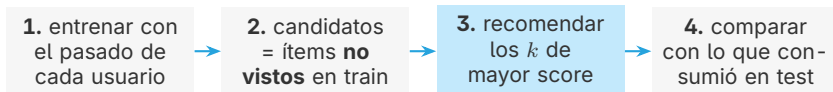
Evaluar antes de modelar

El split también tiene tiempo



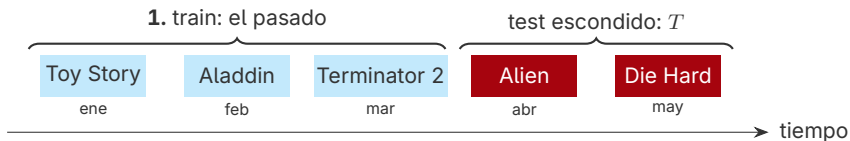
Leave-last-out por usuario: las últimas interacciones de cada usuario van a test. Un split aleatorio de interacciones deja que el modelo aprenda con preferencias **futuras** (mismo principio que la S3).

Protocolo de evaluación top-N



- las métricas se calculan **por usuario** y luego se promedian;
- filtrar lo ya visto es obligatorio: lo que ya consumió no es una recomendación útil y ocupa posiciones del top- k ;
- todos los modelos se evalúan con el **mismo** split, los mismos usuarios y el mismo k .

El protocolo paso a paso: Elena



Película	score	2. ¿candidata?
Toy Story	1,00	ya vista → fuera
Aladdin	0,90	ya vista → fuera
Die Hard	0,90	3. #1
Terminator 2	0,88	ya vista → fuera
Lion King	0,83	3. #2
Alien	0,71	

4. Top - 2 = [Die Hard ✓, Lion King ×]
frente a $T = \{\text{Alien, Die Hard}\}$:
 $P@2 = 1/2 \cdot R@2 = 1/2 \cdot \text{NDCG}@2 = 0,61$.

Sin el paso 2

Top - 2 = [Toy Story, Aladdin]: ya las vio.
0 aciertos y dos posiciones perdidas.

Se repite para **cada** usuario y se promedia.

Precision@k y Recall@k

$$P@k = \frac{|\text{top-}k_u \cap T_u|}{k}$$

$$R@k = \frac{|\text{top-}k_u \cap T_u|}{|T_u|}$$

- $\text{top-}k_u$: lista recomendada a u ;
- T_u : ítems de test que u consumió;
- se promedian sobre los usuarios de test.

Ejemplo: $k = 10$, $|T_u| = 4$

2 aciertos en el top-10:

$P@10 = 2/10 = 0,2 \cdot R@10 = 2/4 = 0,5$.

Recall@k es el mismo concepto de la S1 (fraude), ahora por usuario.

NDCG@k: el orden importa

$$\text{DCG}@k = \sum_{j=1}^k \frac{rel_j}{\log_2(j+1)}$$

$$\text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}$$

- $rel_j = 1$ si el ítem en la posición j está en T_{ui} ;
- acertar en la posición 1 pesa más que en la 10;
- IDCG: el mejor orden posible; NDCG $\in [0, 1]$.

Ejemplo: Dos listas con 1 acierto

Acierto en la posición 1: $\text{DCG} = 1$.
Acierto en la posición 5: $\text{DCG} = 1 / \log_2 6 \approx 0,39$.
Precision@10 es igual en ambas;
NDCG no.

Métricas paso a paso: Elena con $k = 2$

Lo que Elena vio después (test): $T = \{\text{Alien}, \text{Die Hard}\}$. Dos modelos proponen:

Modelo	Top-2	P@2	R@2	DCG@2	NDCG@2
A	Lion King, Alien	1/2	1/2	$0 + \frac{1}{\log_2 3} = 0,63$	$0,63/1,63 = \mathbf{0,39}$
B	Die Hard , Lion King	1/2	1/2	$\frac{1}{\log_2 2} + 0 = 1$	$1/1,63 = \mathbf{0,61}$

IDCG@2 = la mejor lista posible (los 2 aciertos arriba):

$$\frac{1}{\log_2 2} + \frac{1}{\log_2 3} = 1 + 0,63 = 1,63.$$

Idea clave: **Lectura**

Mismo número de aciertos, mismo P y R. B acierta **arriba**: NDCG lo premia.

Más allá del acierto

Métrica	Qué mide	Cálculo simple
Coverage	fracción del catálogo que aparece en algún top- k	$ \bigcup_u \text{top-}k_u /n$
Novedad	qué tan poco popular es lo recomendado	popularidad media de los ítems
Diversidad	qué tan distintos son los ítems de una lista	$1 - \text{similitud media}$

Idea clave: **Lectura**

Un modelo con mejor NDCG puede recomendar casi siempre los mismos 50 ítems. Se reportan acierto **y** cobertura.

¿Por qué los números serán tan pequeños?

En MovieLens cada usuario tiene **5** películas de test entre ≈ 3.600 candidatas, y vemos solo **10**. ¿Cuánto saca una lista **al azar**?

Métrica	Azar	Cuenta
aciertos por lista	0,014	$10 \times 5 / 3.600$
Precision@10	0,0014	aciertos / 10
Recall@10	0,0028	aciertos / 5
NDCG@10	0,002	con descuento, $\div \text{IDCG} = 2,95$

Comparar **la misma métrica**: NDCG 0,06 frente a 0,002 $\Rightarrow \approx 25$ veces mejor que el azar.

Idea clave: **Cómo leer los resultados**

NDCG no es una «nota sobre 100»: se compara la **mejora frente al baseline**, con el mismo protocolo.

Un fallo no siempre es un fallo

El test sabe qué vio después, no todo lo que le habría gustado.

Práctica: ¿cuál desplegaría?

Taller / tarea: Dos modelos, mismo protocolo

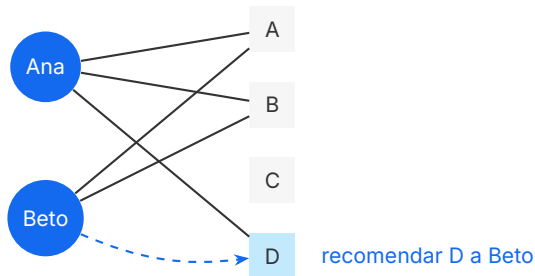
	NDCG@10	Recall@10	Coverage
Modelo A	0,32	0,18	3 %
Modelo B	0,29	0,16	25 %

1. ¿Cuál despliega para la página de inicio de usuarios frecuentes?
2. ¿Y para un catálogo que quiere vender la cola larga?
3. ¿Qué baseline debe aparecer en la tabla antes de decidir?

Filtrado colaborativo y factores latentes

La idea colaborativa

Usuarios con historias parecidas comparten gustos.



No hace falta saber nada del contenido de los ítems: solo quién consumió qué.

Vecindarios: item - item

$$\text{sim}(i, j) = \frac{\mathbf{r}_{\cdot i}^\top \mathbf{r}_{\cdot j}}{\|\mathbf{r}_{\cdot i}\| \|\mathbf{r}_{\cdot j}\|}$$

$$\text{score}(u, i) = \sum_{j \in \text{vistos}(u)} \text{sim}(i, j)$$

Ventajas

- simple e interpretable («porque viste X»);
- no hay que entrenar parámetros.

Límites

- matriz $n \times n$ de similitudes;
- con mucha sparsity, pocos vecinos fiables;
- no generaliza más allá de co-ocurrencias.

Item - item en el mini-mundo

1. Similitud = mismas personas

Columnas 0/1 (quién la vio):

T2 = {Carla, Diego, Elena} · Alien = {Carla, Diego}

$$\text{sim}(T2, \text{Alien}) = \frac{2}{\sqrt{3}\sqrt{2}} = 0,82$$

Lion King = {Ana} · Toy Story = {Ana, Beto, Elena}

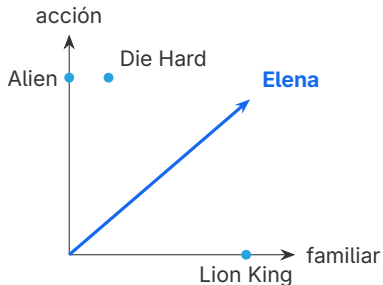
$$\text{sim}(\text{TS}, \text{LK}) = \frac{1}{\sqrt{3}\sqrt{1}} = 0,58$$

2. Score = suma con lo que Elena vio

	TS	Ala	T2	score
Lion King	0,58	0,58	0	1,15
Alien	0	0	0,82	0,82
Die Hard	0	0	0,58	0,58

Top -2: Lion King, Alien. Ningún género participó.

Factores latentes: la intuición



Los nombres «familiar» y «acción» los pusimos nosotros: el modelo aprende los números, no los nombres.

Cada persona y cada película = un vector de $f = 2$ números. Score = **producto punto**:

	$\mathbf{v}_i$	$\mathbf{u}_{\text{Elena}}^\top \mathbf{v}_i$
Lion King	(0,9; 0,0)	0,83
Alien	(0,0; 0,9)	0,71
Die Hard	(0,2; 0,9)	0,90

$\mathbf{u}_{\text{Elena}} = (0,92; 0,79)$: le gustan ambas cosas.

Factores latentes: $R \approx UV^\top$

Qué buscamos: un vector de f números por **usuario** (filas de U) y uno por **película** (columnas de $V^\top$). Esas son las incógnitas que aprende ALS.

U (usuarios $\times f$)									
Ana	0,9	0,1	$\times$	TS	LK	Ala	T2	Alien	DH
Beto	0,8	0,0		1,0	0,9	0,8	0,1	0,0	0,2
Carla	0,1	0,9		0,1	0,0	0,2	1,0	0,9	0,9
Diego	0,0	0,8		$V^\top$ ($f \times$ películas)					
Elena	0,92	0,79							

$$\hat{r}_{\text{Elena, DH}} = \mathbf{u}_{\text{Elena}}^\top \mathbf{v}_{\text{DH}} = 0,92 \cdot 0,2 + 0,79 \cdot 0,9 \approx 0,90$$

- multiplicar $UV^\top$ da un score para **todas** las celdas de R , también las vacías: esas son las recomendaciones;
- los números se **aprenden** para que $UV^\top$ se parezca a R donde sí hay datos; aquí los pusimos a mano y les dimos nombre («familiar», «acción»), el modelo no lo hace;
- en vez de $m \times n$ celdas guardamos $(m + n) \times f$ números ($f \ll n$).

La función objetivo

Incógnitas: todos los vectores de usuarios $\mathbf{u}_u$ y de películas $\mathbf{v}_i$. **Criterio:** que su producto punto se parezca a los ratings conocidos.

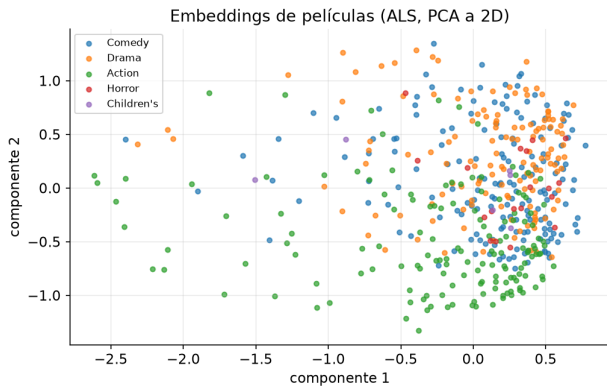
$$\min_{U,V} \sum_{(u,i) \in \Omega} (r_{ui} - \mathbf{u}_u^\top \mathbf{v}_i)^2 + \lambda \left(\sum_u \|\mathbf{u}_u\|^2 + \sum_i \|\mathbf{v}_i\|^2 \right)$$

- solo se suman las celdas **observadas** Ω ;
- la SVD clásica haría lo mismo, pero exige la matriz **completa**, y aquí $\approx 95\%$ está vacío;
- ALS es el algoritmo que resuelve este mínimo (sección siguiente);
- λ : regularización L2, como en Ridge.

Sin regularización

Con pocos ratings por usuario, los vectores se ajustan a esos pocos puntos y sobreajustan.

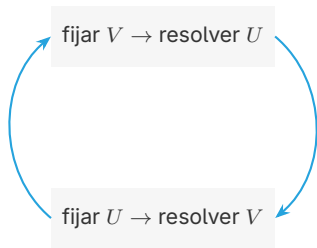
Qué capturan los embeddings (opcional)



- ítems cercanos en el espacio latente se consumen juntos;
- suelen agruparse por género o época sin que nadie lo indique;
- **punto a la S5:** estos vectores se pueden reducir y agrupar.

ALS: Alternating Least Squares

Alternar para que cada paso sea fácil



- el objetivo **no** es convexo en (U, V) a la vez;
- pero sí lo es en U con V fijo, y en V con U fijo;
- cada paso tiene solución cerrada;
- se repite unas 10–20 iteraciones.

Recordemos Ridge

Regresión Ridge clásica

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$$

- X : features (una fila por ejemplo);
- y : target;
- β : coeficientes a aprender;
- λ : encoge β para no sobreajustar.

El mismo molde para Elena

Ridge	ALS, usuario Elena
ejemplos	películas
X	vectores v_i (V fijo)
y	1 si las vio, 0 si no
β	u_{Elena}

Con V fijo, aprender el vector de un usuario es **una regresión lineal**.

Cada paso es una regresión Ridge

Con V fijo, cada usuario se resuelve por separado, en paralelo:

$$\mathbf{u}_u = (V_u^\top V_u + \lambda I)^{-1} V_u^\top \mathbf{r}_u$$

- V_u : filas de V de los ítems que u calificó;
- $\mathbf{r}_u$: ratings de u sobre esos ítems;
- lo mismo para cada ítem con U fijo.

Idea clave: **Lectura**

Para el usuario u , las «features» son los vectores de los ítems que vio y el «target» sus ratings: es exactamente Ridge.

ALS para feedback implícito

Con implícito se usan **todas** las celdas, ponderadas por la confianza:

$$\min_{U,V} \sum_{u,i} c_{ui} (p_{ui} - \mathbf{u}_u^\top \mathbf{v}_i)^2 + \lambda \left(\sum_u \|\mathbf{u}_u\|^2 + \sum_i \|\mathbf{v}_i\|^2 \right)$$

$$\mathbf{u}_u = (V^\top C_u V + \lambda I)^{-1} V^\top C_u \mathbf{p}_u$$

- $C_u = \text{diag}(c_{u1}, \dots, c_{un})$: confianza de u en cada ítem;
- $\mathbf{p}_u$: vector de preferencias (1 si interactuó);
- los ítems no vistos empujan el score hacia 0 con peso bajo (confianza 1).

Un paso de ALS para Elena

1. Qué sabemos: los vectores de las películas V (fijos en este paso).

2. Qué buscamos: los 2 números de Elena, u_{Elena} .

3. La pregunta (una regresión Ridge): ¿qué u hace que

■ $u \cdot v_i \approx 1$ en lo que vio (peso $1 + \alpha = 11$),

■ $u \cdot v_i \approx 0$ en lo que no vio (peso 1)?

4. Se resuelve con la fórmula de la slide anterior ($\alpha = 10$, $\lambda = 0,1$):

$$u_{\text{Elena}} = (0,92; 0,79)$$

5. Luego se fija U , se recalcula cada película igual, y se repite.

Película	¿vio?	peso	$u \cdot v_i$
Toy Story	1	11	1,00
Aladdin	1	11	0,90
Terminator 2	1	11	0,88
Die Hard	0	1	0,90
Lion King	0	1	0,83
Alien	0	1	0,71

Idea clave: **Lectura**

Lo visto queda cerca de 1. Las no vistas que quedan **altas** son las recomendaciones: Die Hard, Lion King.

El truco que lo hace escalable

$$V^T C_u V = \underbrace{V^T V}_{\text{una vez para todos}} + \underbrace{V^T (C_u - I) V}_{\text{solo ítems vistos por } u}$$

Por qué importa

$C_u - I$ es cero en los ítems no vistos. Cada usuario cuesta proporcional a **sus** interacciones, no al tamaño del catálogo. El costo total es lineal en el número de interacciones.

Sin este truco, ALS implícito costaría $O(m \cdot n \cdot f^2)$ por iteración: inviable con catálogos grandes.

Hiperparámetros de ALS

Hiperparámetro	Si sube	Riesgo
factores f	más capacidad	sobreajuste, más costo
regularización λ	vectores más pequeños	subajuste si es muy alta
confianza α	más peso a lo observado	ignora la señal de lo no visto
iteraciones	mejor ajuste a train	poco beneficio tras 15

Idea clave: **Cómo se eligen**

Con **NDCG@k en validación** (leave-last-out), no con el error de reconstrucción. Es la misma lógica de búsqueda de hiperparámetros de las S1 y S2.

Recomendar con ALS

Top- k para un usuario

1. $\text{scores} = V \mathbf{u}_u$ (todos los ítems);
2. quitar los ítems ya vistos;
3. ordenar y tomar los k mayores.

Ítems similares

Vecinos de $\mathbf{v}_i$ por similitud coseno.

Límite: cold start

Un usuario o ítem nuevo no tiene vector: ALS no puede puntuarlo hasta re-entrenar. Se resuelve con contenido (sección siguiente).

ALS implícito en código

```
1 def als_half_step(X, Y, lam, alpha):
2     """Solve one side (rows of X) with the factors Y of the other side fixed."""
3     YtY = Y.T @ Y                                #  $V^T V$ : once for everybody
4     out = np.zeros((X.shape[0], Y.shape[1]))
5     for row in range(X.shape[0]):
6         idx = X.indices[X.indptr[row]:X.indptr[row + 1]]    # items seen
7         Yu = Y[idx]
8         A = YtY + alpha * (Yu.T @ Yu) + lam * np.eye(Y.shape[1])
9         b = (1.0 + alpha) * Yu.sum(axis=0)                  #  $V^T C_u p_u$ 
10        out[row] = np.linalg.solve(A, b)
11    return out
12
13 for _ in range(15):
14     U = als_half_step(X, V, lam, alpha)                    # users, items fixed
15     V = als_half_step(X.T.tocsr(), U, lam, alpha)           # items, users fixed
```

Alternativa de producción: `implicit.als.AlternatingLeastSquares(factors=64, regularization=0.05)`.

Pausa: verifique

Taller / tarea: Colaborativo y ALS

1. ¿Por qué item-item sube Alien para Elena si nadie le dijo que Alien es de ciencia ficción?
2. En el paso de ALS para Elena, ¿qué hace el término $\alpha V_u^\top V_u$?
3. Mañana se registra Fabio: ¿qué vector u tiene en ALS?

Respuestas: Carla y Diego vieron T2 y Alien · da peso $1 + \alpha$ a lo visto · ninguno hasta reentrenar (cold start).

Híbridos: contenido + colaborativo

Recomendación basada en contenido

- **perfil de ítem:** géneros, año, etiquetas (vector x_i);
- **perfil de usuario:** promedio de los perfiles de los ítems que le gustaron;
- **score** = similitud coseno entre perfiles.

Elena: perfil = Animación 2, Familiar 2, Musical 1, Acción 1, Sci-Fi 1.
Lion King 0,87 · Alien 0,21 · Die Hard 0,21.
Ningún otro usuario participó.

Fortalezas

Funciona con **ítems nuevos** y no necesita a otros usuarios. Es explicable: «porque te gustan los dramas».

Debilidad

Sobre-especializa: recomienda más de lo mismo.

¿Por qué combinar?

	Colaborativo (ALS)	Contenido
Captura	gustos sutiles y co-ocurrencias	atributos explícitos del ítem
Cold start	falla con usuarios o ítems nuevos	funciona con ítems nuevos
Sorpresa	puede descubrir lo inesperado	recomienda lo parecido
Datos	solo interacciones	metadatos del catálogo
Elena	sube Alien y Die Hard (sí los vio después)	Lion King (más animación)

Idea clave: Híbrido

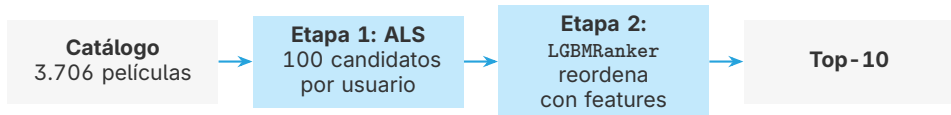
Combinar ambas señales para cubrir las debilidades de cada una.

Estrategias híbridas

Estrategia	Cómo combina	Ventaja	Riesgo
Ponderado	$\beta s_{\text{ALS}} + (1 - \beta) s_{\text{cont}}$	simple	elegir β
Switching	contenido si el usuario tiene poca historia, ALS si no	resuelve cold start	umbral arbitrario
Feature augm.	la salida de un modelo es feature del otro	aprovecha ambas señales	más complejo
Cascada / re-ranking	un modelo genera candidatos y otro los reordena	escala y es preciso	dos etapas que mantener

Burke (2002), *Hybrid Recommender Systems: Survey and Experiments*.

Re-ranking con learning to rank



- features: score ALS, popularidad, coincidencia de géneros usuario–ítem, año, edad y ocupación del usuario;
- objetivo *lambdarank*: optimiza directamente NDCG dentro de cada usuario;
- es el boosting de la S2, ahora como **ranker**.

Cómo se entrena el ranker: vuelve a ser una tabla

Cada candidata que propone ALS se vuelve **una fila**, como en la S1–S2:

¿quién?		features X : lo que sabe el modelo				label y
usuario	película	score ALS	puesto en ALS	similitud de géneros	popularidad	¿la vio después?
Elena	Die Hard	0,90	1	0,21	baja	sí (1)
Elena	Lion King	0,83	2	0,87	baja	no (0)
Elena	Alien	0,71	3	0,21	media	sí (1)

- LGBMRanker (boosting de la S2) aprende de X a y ; en MovieLens, 100 filas por usuario;
- «¿la vio después?» sale de **validación**; test no se toca;
- solo compara filas del **mismo usuario** («grupo»): aprende a poner arriba las de label 1.

Comparar estrategias con el mismo protocolo

Estrategia	NDCG@10	Recall@10	Coverage
Aleatorio	0,002	0,003	100 %
Popularidad	0,035	0,043	3 %
Item - ítem	0,055	0,065	9 %
Contenido (géneros)	0,010	0,012	79 %
ALS	0,062	0,077	30 %
Híbrido ponderado ($\beta = 0,9$)	0,062	0,076	31 %
Re-ranking (ALS + LGBMRanker)	0,063	0,079	43 %

Idea clave: Reglas de comparación

Mismo split leave-last-out (5.656 usuarios de test, últimas 5 películas), mismo $k = 10$ e ítems vistos filtrados. Resultados del notebook de la sesión.

Práctica: elija una estrategia

Taller / tarea: **Tres escenarios**

1. Una plataforma de noticias con miles de artículos nuevos cada día.
2. Un e-commerce que recibe muchos usuarios nuevos por campañas.
3. Un catálogo de películas estable, con usuarios con mucha historia.

Para cada caso: ¿colaborativo, contenido o qué híbrido? ¿Qué métrica reportaría además de NDCG?

Explicar un ranking: SHAP

¿Por qué explicar un recomendador?

- **confianza:** «¿por qué estoy viendo esto?»;
- **depuración:** detectar fugas y features que no deberían pesar;
- **equidad:** ¿la edad o el género deciden lo que se muestra?;
- **regulación:** transparencia de decisiones automatizadas.

Idea clave: Qué se explica

Los factores latentes de ALS no tienen nombre. Se explica el **ranker** de la etapa 2, que sí usa features con significado.

La idea de SHAP: repartir un premio con justicia

Ana y Beto juegan trivia en equipo y ganan **100 puntos**. ¿Cuánto mérito tiene cada uno?

Lo que sacarían

Equipo	Puntos
nadie	0
solo Ana	60
solo Beto	20
Ana y Beto	100

Armamos el equipo de a uno: ¿cuánto sube al unirse cada uno?

Orden	Ana	Beto
entra Ana, luego Beto	$60 - 0 = 60$	$100 - 60 = 40$
entra Beto, luego Ana	$100 - 20 = 80$	$20 - 0 = 20$
promedio	70	30

Idea clave: Valor de Shapley

Mérito de cada jugador = los puntos que agrega al **unirse** al equipo, **promediados** sobre todos los órdenes de entrada. Y siempre suma el total: $70 + 30 = 100$.

En SHAP: jugadores = features · premio = cuánto se aleja el score del promedio.

Lo mismo con el ranker: Elena–Die Hard

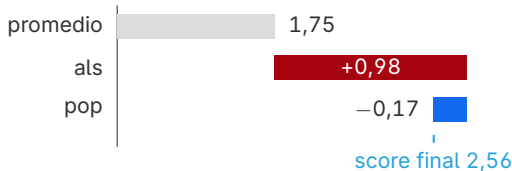
Ranker de juguete con 2 features: $f(\text{als}, \text{pop}) = 2 \text{ als} + \text{pop} + \text{als} \cdot \text{pop}$.
«Apagar» una feature = ponerla en su valor promedio (0,5).

Features encendidas	als, pop	score
ninguna (score promedio)	$0,5 \cdot 0,5$	1,75
solo als	$0,9 \cdot 0,5$	2,75
solo pop	$0,5 \cdot 0,4$	1,60
ambas (Elena–DH)	$0,9 \cdot 0,4$	2,56

$$\phi_{\text{als}} = \frac{1}{2} [(2,75 - 1,75) + (2,56 - 1,60)] = +0,98$$

$$\phi_{\text{pop}} = \frac{1}{2} [(1,60 - 1,75) + (2,56 - 2,75)] = -0,17$$

Lectura: Die Hard sube por el score de ALS y baja un poco por ser poco popular.



Se cumple siempre

$$1,75 + 0,98 - 0,17 = 2,56$$

El valor de Shapley

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} \left[f(S \cup \{j\}) - f(S) \right]$$

- F : todos los jugadores (features);
- S : los que **ya estaban** en el equipo cuando entra j ;
- $f(S \cup \{j\}) - f(S)$: cuánto sube el score cuando j se une a S ;
- la fracción: peso de ese S (qué parte de los órdenes lo tienen antes de j);
- ϕ_j : promedio ponderado de los aportes.

Lundberg y Lee (2017), *A Unified Approach to Interpreting Model Predictions*.

Ejemplo: Trivia: $j = \text{Beto}$

S	aporte	peso
{Ana}	$100 - 60 = 40$	$\frac{1}{2}$
$\emptyset$ (nadie)	$20 - 0 = 20$	$\frac{1}{2}$
$\phi_{\text{Beto}} = \frac{1}{2} \cdot 40 + \frac{1}{2} \cdot 20 = \mathbf{30}$		

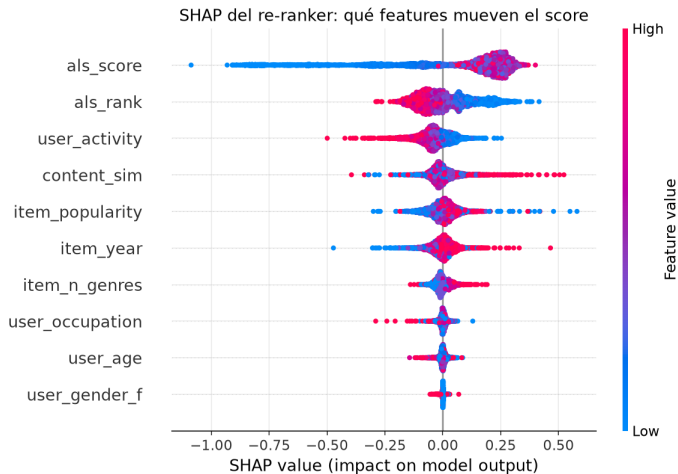
$$f(x) = \phi_0 + \sum_{j=1}^{|F|} \phi_j$$

Eficiencia

La explicación suma **exactamente** el score del modelo: ϕ_0 es el score promedio y cada ϕ_j lo empuja hacia arriba o hacia abajo.

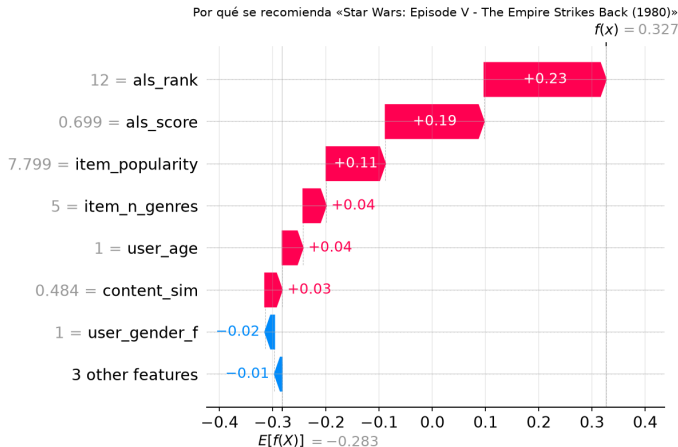
- **simetría**: dos features que aportan lo mismo reciben lo mismo;
- **nulidad**: una feature que no cambia nada recibe 0;
- **TreeSHAP**: cálculo exacto y rápido para modelos de árboles (LightGBM, XGBoost).

Explicación global del ranker



- cada punto = un par usuario-ítem;
- eje x = ϕ_j (empuje al score);
- color = valor de la feature;
- orden = importancia global.

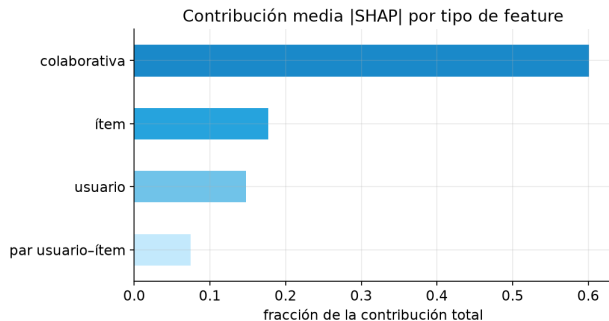
Explicación local: un usuario y una película



Se lee de abajo hacia arriba: parte de $E[f(x)] = \phi_0$ (score promedio); cada barra roja **sube** y cada azul **baja**; arriba queda $f(x)$, el score de esta película para **este** usuario. Es la tabla del premio, con 10 features.

Así se construye el «porque te gustó...» que ve el usuario.

Contribuciones de usuario frente a ítem



Se suman los $|\phi_j|$ por tipo de feature:

- **colaborativa** (score ALS);
- **ítem** (popularidad, año, géneros);
- **usuario** (edad, ocupación);
- **par** (coincidencia de géneros).

¿El modelo personaliza o solo empuja lo popular?

Cuidados al interpretar SHAP

SHAP explica el modelo, no el mundo

Una contribución alta dice que el modelo usa esa feature, no que la feature cause la preferencia.

- features correlacionadas se reparten el crédito: leer por grupos;
- el valor depende del modelo y del fondo de referencia elegido;
- también aplica a la S3: qué lags y variables de calendario mueven el pronóstico de un HGBR.

Hands-on y cierre

Hands-on de la sesión

Caso: MovieLens 1M (GroupLens) convertido a feedback implícito.

1. **ALS sobre datos implícitos:** entrenar y comparar contra popularidad (NDCG@10, Recall@10).
2. **SHAP:** contribuciones de las features de usuario e ítem en el re-ranker.
3. **Top-N con estrategias híbridas:** ponderado frente a re-ranking, mismo protocolo.

Formato cuaderno guiado: 15 ejemplos resueltos paso a paso (prediga, siga la cuenta, discuta), primero sobre el mini-mundo y luego sobre MovieLens. El código pesado va en una celda plegada.

Herramienta: **Notebook**

[04_ml_recomendadores_shap.ipynb](#) (local con uv o en Colab)

Lo que el notebook debe demostrar

Salida	Pregunta que responde
Popularidad frente a ALS	¿la personalización supera al baseline serio?
Búsqueda de f , λ , α	¿qué configuración gana en NDCG@10 de validación?
Tabla de estrategias	¿el híbrido mejora cobertura sin perder acierto?
SHAP global y local	¿qué features dominan el ranking y por qué?

Glosario mínimo

- **Contenido / colaborativo**: score a partir de atributos del ítem / a partir de lo que vieron otros usuarios.
- **Matriz de utilidad / sparsity**: usuarios $\times$ ítems / fracción de celdas observadas.
- **Feedback implícito**: interacciones sin negativos; preferencia p_{ui} y confianza c_{ui} .
- **Factores latentes**: vectores $\mathbf{u}_u, \mathbf{v}_i$ con $\hat{r}_{ui} = \mathbf{u}_u^\top \mathbf{v}_i$.
- **ALS**: alternar regresiones Ridge para usuarios e ítems.
- **Leave-last-out**: test = últimas interacciones de cada usuario.
- **Precision@k, Recall@k, NDCG@k, coverage**: métricas de ranking y de catálogo.
- **Híbrido / re-ranking**: combinar contenido y colaborativo; candidatos + ranker (grupo = usuario, lambdarank).
- **Valor de Shapley / TreeSHAP**: cuánto sube el score cuando cada feature se une, promediado sobre todos los órdenes de entrada (coaliciones).

Preguntas que ya debe poder responder

1. ¿Qué evidencia usa un recomendador de contenido y cuál uno colaborativo?
2. ¿Por qué un hueco en la matriz de utilidad no es un cero?
3. ¿Qué cambia en la pérdida de ALS cuando el feedback es implícito?
4. ¿Por qué ALS alterna en lugar de optimizar U y V a la vez?
5. ¿Por qué no se hace un split aleatorio de interacciones?
6. ¿Qué mide NDCG@k que Precision@k no mide?
7. ¿Cuándo usaría un híbrido y cuál?
8. ¿Qué garantiza la propiedad de eficiencia de SHAP?

La frase técnica que debe sobrevivir

Un recomendador merece confianza cuando se evalúa con un split que respeta el tiempo, supera al baseline de popularidad en métricas de ranking, cuida la cobertura y puede explicar **por qué** ordenó así.

Transición a la siguiente sesión

Hoy usuarios e ítems se convirtieron en **vectores**.

¿Qué estructura aparece cuando no hay etiquetas?

Idea clave: **Sesión 5**

Reducción de dimensión (PCA, t-SNE, UMAP) y clustering (K-Means, DBSCAN, HDBSCAN).

Gracias por su atención

¿Preguntas, comentarios o discusión?



Aprendizaje Automático
SI7009 · Universidad EAFIT

